Self Integration Architecture Operational Specification: Decomposing B1.05 *Self as Integrated Whole* by Formalizing How the Self Holds Multiple Aspects Coherently Through Substrate-Resident Integration Architecture

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 8, 2026

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in *The Instinct/Reasoning Separation Outside the Model* (Li, April 2026), the second paper in the CKS theory series following *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one operational variant of B1.05 (Self as integrated whole) — the **Self integration architecture operational specification** — which articulates how the Self holds multiple aspects coherently through substrate-resident integration architecture, what mechanisms that architecture comprises, and how those mechanisms preserve the inherited commitments from Paper 1 and the structural commitments from Paper 2.

Abstract

Paper 2 names the Self as the integrated whole that holds multiple aspects as facets of one CKS-governed intelligence (B1.05; B2.20). The companion variant B2.20 formalizes that commitment at the level of foundational architectural framing. This note formalizes the next layer down: the **operational specification** of how integration is held — the substrate-resident integration architecture per A2.46 comprising aspect coexistence governance, Self-level operations spanning aspects, aspect facet character maintenance per B2.20, cross-aspect conflict handling per A1.03, cross-aspect integration rules using Pattern A/B/C per A2.92–A2.94, Self-level decisions, and cross-level access configuration per B1.19. The seven mechanisms operate together; each is independently governable; their substrate-resident character is what distinguishes Self integration in CKS from the implicit integration of conventional AI multi-component architectures. The note states the specification, articulates what makes it distinctive, names the cognitive analog, identifies the inherited commitments, draws operational implications, and bounds the limits.

1. Why Self integration architecture needs its own operational specification

B1.05 names the Self as the integrated whole holding multiple aspects as facets of one intelligence. B2.20 formalizes that commitment at the level of architectural framing. Neither alone specifies *how integration is operationally held*. The distinction matters: a system can satisfy B2.20's framing (the Self holds aspects as facets, not subsystems) while leaving the integration mechanism implicit — configured in application code, distributed across runtime middleware that is not architecturally describable. Paper 2's commitment is stronger: integration is itself substrate-resident, governable, inspectable, and auditable.

Conventional AI multi-component architectures rarely treat integration as a governable artifact in its own right. They treat it as engineering — code wires components together, configuration tunes the wiring, runtime middleware coordinates. The integration is implementation, not content. CKS Selves cannot rest there, because every Paper 1 commitment (path retraceability, conflict-as-first-class, human-governed) requires that integration be inspectable, modifiable, and overridable by the same authority architecture that governs the substrate it integrates over.

This note is the second of five decomposing B1.05; the others address the integrated whole at foundational level (B2.20), the Self–aspect content-domain relationship (B2.22), Self-level instinct/reasoning configuration (B2.23), and Self-level inheritance of Paper 1 commitments (B2.24).

2. The integration architecture, specified

A Self's **integration architecture** is the substrate-resident operational specification per A2.46 that determines how the Self's multiple aspects are held coherently. The architecture comprises seven mechanisms, each authored as orchestration content per A2.04, each recorded with provenance per A2.40, each independently inspectable, modifiable, and overridable per A2.01–A2.03.

Mechanism 1 — Aspect coexistence governance. Rules specify how aspects coexist within the Self. Multiple aspects operate concurrently: a Self may run its sports-mode aspect and its study-mode aspect at the same time, possibly over overlapping cells (cells participate in multiple aspects per B1.04). Coexistence rules govern what happens when two aspects ask the same cell different pattern questions, when aspects trigger overlapping orchestration paths, and when aspect-level outputs must be sequenced or interleaved at Self level. The rules are substrate-resident; they are not implicit in code that happens to run aspects sequentially.

Mechanism 2 — Self-level operations spanning aspects. The Self has its own operations distinct from any single aspect. Self-level reasoning may span multiple aspects via the content-domain relationship formalized in B2.22 — the Self treats aspect outputs as content over which it reasons, and Self-level outputs integrate aspect content into a Self-level result. The Self is itself a reasoning locus operating over its aspects, architecturally distinct from "aspect produces output, Self exposes it."

Mechanism 3 — Aspect facet character maintenance. Operational mechanisms preserve the aspects-as-facets character per B2.20. Aspects are not treated as subsystems by the Self. The integration architecture does not turn aspects into modules whose internal state is the Self's to read or write at will, nor does it treat aspects as services the Self calls. Aspects retain their level-distinct scope per B2.07. The mechanism is operational, not merely declarative: the rules that govern Self–aspect interaction must keep the Self from collapsing facet character into subsystem character even under operational pressure.

Mechanism 4 — Cross-aspect conflict handling per A1.03. When aspects produce conflicting outputs at Self level — aspect P says "X," aspect Q says "not-X" — the Self **registers the conflict as a first-class architectural artifact**. It does not auto-resolve. Conflicts are surfaced for human governance through the same conflict-preservation mechanism that operates at substrate level under A1.03; the Self level does not get to collapse them just because it is the integration locus. The conflict-as-first-class commitment scales up to the Self.

Mechanism 5 — Cross-aspect integration rules using Pattern A/B/C per A2.92–A2.94. Rules specify how aspect outputs combine for Self-level outputs. The three composition patterns from Paper 1 (A1.16) apply at Self–aspect scope:

- *Pattern A (consultation).* The Self consults aspects as inputs to a Self-level reasoning operation, then writes its output under a Self-level orchestration rule.
- *Pattern B (derived view).* The Self derives a synthesis or projection from multiple aspect outputs, without that derived view becoming authoritative over the aspects it derives from.
- *Pattern C (separate concerns).* Aspects operate independently for distinct concerns, with the Self performing separate Self-level reasoning that does not require integration.

The patterns are architectural positions, not deployment recipes. A single Self may use all three across its different aspects.

Mechanism 6 — Self-level decisions. Decisions made at the Self level about aspect interactions, lifecycle, introduction, or dissolution. These are governed per A1.01 — humans hold the rights to inspect, modify, and override Self-level decisions and the rules that produce them. Many Selves will configure such decisions as rare events triggered by specific governance moments.

Mechanism 7 — Cross-level access configuration per B1.19. The Self may access cells directly, bypassing aspects, under specified rules. The integration architecture configures when this is permitted, under which rules, with what provenance recording. Integration is not only about how aspects combine at Self level but also about how the Self relates to cells across aspects.

The seven mechanisms are substrate-resident per A2.46. Humans inspect them per A2.01, modify them per A2.02, override them per A2.03, author the rules that constitute them per A2.04. Integration events are recorded per A2.40 with the six-field provenance vocabulary.

3. What makes this architecture distinctive

Conventional AI multi-component architectures integrate components implicitly. Code wires components together; configuration tunes the wiring; runtime middleware coordinates. The integration is engineering, not content. When integration changes — a new component is added, a routing rule is altered, a coordination policy is updated — the change is a deployment event. There is no inspectable artifact that says "this is how integration was, here is what changed, here is who authorized the change, here is the rule that governs it now." The integration *behaves*; it does not *exist as inspectable content*.

CKS Self integration is the opposite. The integration architecture is substrate-resident authoritative content per A2.46. Its mechanisms are governable artifacts. Changes are governance events recorded with provenance per A2.40. Given a Self-level output, the mechanisms that produced it are reconstructible from substrate content rather than reverse-engineered from logs of side effects.

The treatment is consequential in three ways. First, Self integration becomes auditable in the same sense as cell-level work: a regulator, an auditor, or an internal reviewer can read the integration architecture as content. Second, integration becomes evolvable through the same mechanisms as substrate content: directed selection per B1.14, action-feedback per B1.15, vertical evolution per B1.16. Third, cross-aspect conflicts

cannot be silently merged under Self-level integration logic — A1.03's first-class commitment holds at Self scope because the integration architecture surfaces conflicts rather than collapsing them.

4. The cognitive analog as conceptual scaffold

Self integration architecture parallels human cognitive integration. A psychologically integrated person operates through specific cognitive integration mechanisms — executive function that coordinates competing demands, narrative integration that weaves experiences into a coherent self-story, attentional control that allocates processing across modalities, value integration that resolves competing motivations under reflection. Biology has loose analog in nervous-system integration across distributed neural systems.

The analog functions as conceptual scaffold per Paper 2's biology positioning. Readers absorb it quickly because the parallels are intuitive. The architectural substance is CKS's own: governable substrate-resident integration rules, with the seven mechanisms enumerated in §2, recorded with provenance, evolvable under human governance.

5. Inherited Paper 1 and Paper 2 commitments

The integration architecture is recursive in its commitments — every commitment from Paper 1 and from Paper 2's foundational layer holds.

- **A1.01 human-governed.** Integration architecture is inspectable, modifiable, and overridable at any time.
- **A1.03 conflict-as-first-class.** Cross-aspect conflicts at Self level are registered as first-class artifacts; the Self does not auto-resolve.
- **A1.10 determinism contract.** Given a fixed integration architecture and fixed inputs, integration behavior is deterministic in the sense the contract permits.
- **A1.13 composition requirements.** All five composition constraints are satisfied at Self–aspect scope; integration architecture is not a fourth composition primitive.
- **A1.16 / A2.92–A2.94 hybrid composition.** The three composition patterns A/B/C apply directly to the Self–aspect relationship.
- **A2.04 rule authoring.** Integration rules are authored by humans; LLM-drafted rules are admissible only under human authority before they take effect.
- **A2.40 provenance.** Integration events are recorded with the six-field metadata.
- **A2.46 Category 4.** Integration architecture is authoritative substrate content.
- **B2.07 / B2.20.** Integration preserves aspects-as-facets and respects level-distinct scope; aspects do not collapse into Self-level subsystems.

6. Operational implications

Six implications follow.

Deployments configure integration architecture per Self purpose. What aspects the Self holds, how they integrate, what cross-aspect operations are needed, what cross-level access is permitted — all are

deployment decisions made at Self design time and refined through operation. The architecture supports configuration; it does not prescribe a single integration topology.

Integration architecture evolves through directed selection per B1.14. Humans refine integration based on operational experience: tightening coexistence rules when aspects interfere, loosening cross-level access when purpose changes, restructuring Pattern A/B/C assignments when the Self's work mix shifts.

Integration architecture evolves through action-feedback per B1.15. Accumulated Self operations inform refinement under human mediation; integration rules refactor under accumulated operational experience, with humans mediating the refactor.

Vertical evolution per B1.16 modifies integration architecture. When aspects are added, removed, or restructured, the integration architecture is what changes. This is what makes Self-level vertical evolution operationally available.

Cross-partner integration follows authority distribution per A2.47. When multiple human partners hold authority over different aspects of the same Self, the integration architecture specifies how their authorities compose at Self level.

Integration is testable through inspection and replay. A5.16 reproducibility verifies integration determinism: given a fixed integration architecture and fixed inputs, the same trace must reproduce. High-stakes Self-level decisions per B2.05 may use stricter integration patterns (e.g., Pattern A consultation with mandatory conflict registration) for tighter governance.

7. Limits

The integration architecture does several things; equally, it does not do several things.

- **It does not command aspects.** The Self relates to aspects through content-domain per B1.18; integration is not command-and-control.
- **It does not bypass aspect scope.** Integration operates at the Self–aspect boundary, not by reaching into aspect internals; aspects retain level-distinct scope per B2.07.
- **It does not auto-resolve cross-aspect conflicts.** A1.03 first-class preservation holds at Self level.
- **It does not eliminate aspect-level governance.** Aspects remain governed at aspect level; Self integration does not subsume aspect governance.
- **It does not prescribe specific patterns.** Deployments configure Pattern A/B/C assignments per Self purpose.
- **It does not eliminate facet character.** Integration is the mechanism by which facet character is *preserved* under operational pressure, not the mechanism by which it is dissolved.
- **It is not a single mechanism.** Seven mechanisms operate together; isolating any one as "the integration mechanism" misreads the specification.
- **It is not inter-Self.** Paper 2 specifies intra-Self architecture. Multiple Selves and their integration are out of scope for Paper 2.

8. Operational test

A Self instantiates the **Self integration architecture operational specification** if and only if all of the following are true:

1 The seven mechanisms (aspect coexistence governance, Self-level operations spanning aspects, facet character maintenance, cross-aspect conflict handling, cross-aspect integration rules under Pattern A/B/C, Self-level decisions, cross-level access configuration) are each represented as substrate-resident content per A2.46.

2 Each mechanism is independently inspectable, modifiable, and overridable per A2.01–A2.03 within the broader human-governed authority architecture per A1.01.

3 Integration rules are authored under A2.04; integration events are recorded under A2.40 with the six-field provenance vocabulary.

4 Cross-aspect conflicts at Self level are registered as first-class artifacts per A1.03; no Self-level mechanism auto-resolves them.

5 The integration architecture preserves aspects-as-facets per B2.20; aspects are not treated as subsystems by Self-level operations.

6 Cross-level access configuration per B1.19 specifies, in inspectable form, when the Self accesses cells directly bypassing aspects and under what rules.

A Self that fails any of (1)–(6) may be a useful system; it does not instantiate the operational specification this note formalizes.

9. Conclusion: progression through Phase B2

This note is the second of five decomposing B1.05. B2.20 frames the foundational architectural commitment; this note (B2.21) formalizes the operational specification by which integration is held; B2.22 will operationalize the Self–aspect content-domain relationship; B2.23 will specify Self-level instinct/reasoning configuration; B2.24 will verify Self-level inheritance of Paper 1 commitments. Subsequent Phase B2 notes will continue with B1.06's two-layers-within-every-cell decomposition (B2.25–B2.29 and beyond).

Naming Self integration architecture as standalone operational specification matters because the alternative — leaving integration implicit, configured in code, distributed across runtime middleware — would silently break every Paper 1 commitment at Self scope. Path retraceability would not extend to Self-level integration events. Conflict-as-first-class would collapse into Self-level merge logic. Human-governed authority would not extend to integration rules because there would be no integration rules to govern. The operational specification this note formalizes is what keeps Paper 1's commitments architecturally available at the Self scope Paper 2 extends them to. Subsequent work that adopts B1.05, extends it, composes it with adjacent patterns, or argues against it should use the integration architecture vocabulary formalized here. Subsequent work that uses different terms is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance.* April 2026. ORCID: 0009-0004-8065-3235.

Predecessor paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Self Integration Architecture Operational Specification: Decomposing B1.05 Self as Integrated Whole by Formalizing How the Self Holds Multiple Aspects Coherently Through Substrate-Resident Integration Architecture.* May 8, 2026. ORCID: 0009-0004-8065-3235.